

6 Speicher

Speicher bilden eine der zentralen Säulen jeder Computerarchitektur. Sie dienen nicht nur der reinen Datenablage, sondern bestimmen maßgeblich die Leistungsfähigkeit eines Systems. Während die Prozessorlogik (CPU) für die Ausführung von Instruktionen verantwortlich ist, stellen Speicherbausteine sicher, dass die benötigten Daten und Programme rechtzeitig und in der richtigen Form bereitstehen. Geschwindigkeit, Organisation und Anbindung des Speichers beeinflussen daher unmittelbar die Gesamteffizienz eines Systems.

Grundsätzlich unterscheidet man **flüchtige** und **nichtflüchtige** Speicher. Flüchtige Speicher (z. B. DRAM oder SRAM) verlieren ihren Inhalt beim Abschalten der Versorgungsspannung, während nichtflüchtige Speicher (z. B. ROM, EEPROM, Flash) die gespeicherten Informationen dauerhaft bewahren. Diese Eigenschaft ist entscheidend für die Aufgabenteilung zwischen Daten- und Programmspeicher, zwischen kurzfristiger und langfristiger Informationshaltung.

Neben der Unterscheidung nach Flüchtigkeit spielt auch die **Zugriffszeit** eine zentrale Rolle. Da Prozessoren im Vergleich zu Speicherbausteinen extrem schnell arbeiten, entsteht eine sogenannte **Geschwindigkeitskluft** („Memory Gap“) zwischen CPU-Takt und Speicherzugriffszeit. Moderne Systeme kompensieren diese Differenz durch mehrstufige **Speicherhierarchien**, bei denen kleine, sehr schnelle Speicher (Register, Cache) den Zugriff auf größere, langsamere Speicher (RAM, Massenspeicher) puffern. Dieses Prinzip ermöglicht es, die mittlere Zugriffszeit drastisch zu reduzieren, ohne auf die hohe Kapazität kostengünstiger Massenspeicher zu verzichten.

Speicher unterscheiden sich zudem in ihrer **Organisation** (Wortbreite, Adressierungsart, Zugriffsmuster) und in der **Art des Zugriffs**. Während z. B. bei klassischen RAM-Zellen wahlfreier Zugriff („Random Access“) möglich ist, erfolgt der Zugriff bei Flash- oder EPROM-Bausteinen blockweise oder seitenweise. Auch die interne Ansteuerlogik (z. B. Paging, Refresh-Zyklen bei DRAM) beeinflusst Geschwindigkeit, Energiebedarf und Komplexität der Speichersteuerung.

Das Zusammenspiel von CPU, Bus-System und Speicherarchitektur bestimmt somit nicht nur die Rechenleistung, sondern auch den Energieverbrauch, die Echtzeitfähigkeit und die Skalierbarkeit eines Systems. In eingebetteten Systemen gewinnt darüber hinaus die Frage an Bedeutung, **welche Daten dauerhaft erhalten bleiben müssen** – beispielsweise Mess-, Kalibrier- oder Log-Daten – und welche nur temporär während der Programmausführung benötigt werden.

In den folgenden Abschnitten werden die wichtigsten Speicherarten vorgestellt: **Festwertspeicher** (ROM, EEPROM, Flash) zur dauerhaften Programmspeicherung, **Schreib-/Lesespeicher** (RAM) für flüchtige Arbeitsdaten, **nichtflüchtige RAM-Varianten** (NVRAM) als Bindeglied zwischen beiden Welten sowie der **Cache** als entscheidende Zwischenstufe zwischen Prozessor und Hauptspeicher.

6.1 Festwertspeicher

Festwertspeicher dienen der **dauerhaften Speicherung von Programmen, Parametern und Kalibrierdaten**. Sie gehören zur Klasse der **nichtflüchtigen Speicher** und behalten ihren Inhalt auch ohne Versorgungsspannung. Ihr wesentliches Merkmal ist, dass der gespeicherte Inhalt entweder gar nicht oder nur eingeschränkt verändert werden kann. Dadurch eignen sie sich ideal zur Ablage von Firmware, Bootloadern oder Systemtabellen, die zuverlässig und unveränderlich im Gerät verbleiben müssen.

Historisch betrachtet sind verschiedene ROM-Varianten (Read Only Memory) entstanden, die sich vor allem durch ihre Programmier- und Löschmechanismen unterscheiden:

- **ROM (Masken-ROM):** Der Inhalt wird bereits während der Fertigung durch fotolithografische Masken festgelegt. ROMs sind äußerst zuverlässig, aber nicht veränderbar – sie werden heute fast ausschließlich in Massenprodukten mit hohem Stückvolumen verwendet (z. B. Mikrocontroller mit fest eingebrannter Firmware).
- **PROM (Programmable ROM):** Ein einmal programmierbarer Speicher, bei dem einzelne Speicherzellen durch gezielte elektrische Zerstörung (z. B. Schmelzen einer Verbindung) programmiert werden. Nach der Programmierung verhält sich der PROM wie ein normales ROM.
- **EPROM (Erasable PROM):** Ein löschbarer Festwertspeicher, dessen Inhalt durch ultraviolettes Licht gelöscht werden kann. EPROMs erkennt man an dem charakteristischen Quarzglasfenster im Gehäuse. Sie wurden früher häufig in Entwicklungsumgebungen verwendet, da sie mehrfach beschreibbar waren – heute sind sie weitgehend durch elektrisch löschbare Varianten ersetzt.
- **EEPROM (Electrically Erasable PROM):** Diese Speicherzellen können elektrisch gelöscht und neu beschrieben werden. Der Schreibvorgang erfolgt byteweise, was flexible, aber relativ langsame Aktualisierungen erlaubt. EEPROMs werden häufig für Konfigurationsdaten, Kalibrierwerte oder Zählerstände eingesetzt, die selten, aber gezielt geändert werden müssen.
- **Flash-Speicher:** Eine Weiterentwicklung des EEPROM-Prinzips, bei dem Löschung und Programmierung blockweise erfolgen. Dadurch sind Flash-Speicher deutlich schneller und besitzen eine wesentlich höhere Speicherdichte. Sie werden heute in nahezu allen Mikrocontrollern als Programmspeicher (Code Flash) eingesetzt und finden sich auch in externen Speichermedien wie SSDs, USB-Sticks oder SD-Karten.

Der interne Aufbau moderner Flash-Zellen basiert meist auf Floating-Gate- oder Charge-Trap-Technologien, bei denen Ladungen in einem isolierten Gate gespeichert werden. Dadurch bleibt der Programmierzustand auch bei fehlender Versorgungsspannung erhalten. Flash-Speicher unterscheiden sich in **NOR-** und **NAND-Architekturen**: NOR-Flash erlaubt den direkten wahlfreien Zugriff auf einzelne Speicherzellen (typisch für Programmspeicher), während NAND-Flash auf sequenziellen Zugriff optimiert ist und daher vorwiegend für Massenspeicher verwendet wird.

Da Schreib- und Löschzyklen die Speicherzellen physikalisch belasten, ist die Lebensdauer von Flash und EEPROM durch die Anzahl der möglichen Schreibvorgänge begrenzt (typisch 10^4 – 10^6 Zyklen). Um diesen Effekt zu kompensieren, werden **Wear-Leveling-Verfahren** eingesetzt, die Schreibvorgänge gleichmäßig auf verschiedene Speicherbereiche verteilen.

i Info: ROM in modernen Systemen

Auch wenn der klassische Masken-ROM in der Bauteilentwicklung seltener geworden ist, wird **ROM-Technologie weiterhin aktiv eingesetzt**. In vielen Mikrocontrollern und SoCs ist ein **interner Boot-ROM** enthalten, der den Startvorgang des Systems sicherstellt. Ein Beispiel ist der **Raspberry Pi Pico** mit dem RP2040-Mikrocontroller:

Sein UF2-Bootloader befindet sich fest im ROM. Das bedeutet:

- Der Bootloader ist **nicht löschbar** und kann nicht versehentlich überschrieben werden.
- Er ist **immer verfügbar**, selbst wenn der Flash-Speicher gelöscht oder beschädigt ist.
- Man aktiviert ihn durch **Drücken des BOOTSEL-Knopfs beim Anschließen an USB**, wodurch der Pico als USB-Laufwerk erscheint.

ROM wird daher überall dort eingesetzt, wo **zuverlässiger, unveränderlicher Code** benötigt wird – etwa für Bootloader, Sicherheitsfunktionen oder Systeminitialisierung.

 Merke: Festwertspeicher (ROM, EEPROM, Flash)

Festwertspeicher sind **nichtflüchtig**, behalten also ihren Inhalt auch ohne Versorgungsspannung.

- **ROM**: nur lesbar, unveränderlich – ideal für feste Firmware.
- **EEPROM**: elektrisch löscht- und schreibbar, flexibel, aber langsam.
- **Flash**: blockweise programmierbar, hohe Speicherdichte, heute Standard für Programmspeicher. Flash basiert auf Floating-Gate-Technologie und nutzt oft **Wear-Leveling**, um die Lebensdauer zu erhöhen.

6.2 Schreib-/Lesespeicher (RAM)

Im Gegensatz zu Festwertspeichern dienen **RAMs (Random Access Memories)** der **temporären Speicherung** von Daten, Variablen und Zwischenergebnissen während der Programmausführung.

Der Name *Random Access* bedeutet, dass jede Speicherzelle direkt und mit nahezu identischer Zugriffszeit adressiert werden kann – unabhängig von ihrer physikalischen Position im Speicher. RAM zählt zu den **flüchtigen Speichern**, d. h. der gespeicherte Inhalt geht beim Abschalten der Versorgungsspannung verloren.

RAMs bilden das eigentliche **Arbeitsgedächtnis eines Mikroprozessorsystems**. Sie werden verwendet, um Programmdaten, Stackinhalte, Puffer und Laufzeitvariablen abzulegen. Je nach Aufbau und Funktionsweise unterscheidet man zwei Haupttypen: **statisches RAM (SRAM)** und **dynamisches RAM (DRAM)**.

Das **SRAM** speichert jedes Bit in einer Flipflop-Struktur aus Transistoren. Der gespeicherte Zustand bleibt stabil, solange die Versorgungsspannung anliegt, ohne dass ein periodisches Auffrischen erforderlich ist. Statische RAMs zeichnen sich durch **kurze Zugriffszeiten, hohe Geschwindigkeit** und **geringe Latenz** aus, benötigt jedoch deutlich mehr Transistoren pro Speicherzelle und ist damit flächen- und kostenintensiver. Daher werden sie vorwiegend in kleineren Speichern eingesetzt, bei denen Geschwindigkeit entscheidend ist – etwa als **Cache** oder **Registerspeicher** in Mikroprozessoren und Mikrocontrollern.

Das **DRAM** hingegen speichert jedes Bit als elektrische Ladung in einem Kondensator, der über einen Transistor zugeschaltet wird. Da diese Ladung mit der Zeit über Leckströme verloren geht, muss der Speicherinhalt in regelmäßigen Abständen aufgefrischt werden – dieser Vorgang wird als **Refresh** bezeichnet. Das DRAM ist wesentlich **kompakter** und **kostengünstiger** als das SRAM und bildet daher den Hauptspeicher in PCs, Notebooks und vielen SoCs. Nachteilig sind die längeren Zugriffszeiten und die Notwendigkeit einer speziellen **Refresh-Logik**, die im Speichercontroller integriert ist.

Moderne DRAMs sind häufig als **SDRAM (Synchronous DRAM)** oder **DDR-SDRAM (Double Data Rate SDRAM)** ausgeführt. Bei SDRAM erfolgt der Datentransfer synchron zum Bustakt, während DDR-Technologien die Übertragungsrate erhöhen, indem Daten sowohl an der steigenden als auch an der fallenden Taktflanke übertragen werden. Damit lassen sich – trotz unverändertem Takt – deutlich höhere Durchsätze erzielen. Aktuelle Systeme verwenden DDR4- oder DDR5-Module mit hohen Bandbreiten und geringerem Energiebedarf, während in Embedded-Systemen meist statische RAMs oder interne Speicherblöcke im SoC integriert sind.

Einige Mikrocontrollerfamilien bieten neben internem SRAM auch externe Speicherinterfaces (z. B. *External Memory Interface, EMI* oder *Flexible Static Memory Controller, FSMC*), über die zusätzliche SRAM- oder DRAM-Bausteine angebunden werden können. Solche Architekturen ermöglichen den Einsatz größerer Arbeitsspeicher, z. B. für Grafikpuffer oder Signalverarbeitung.

 Merke: Schreib-/Lesespeicher (RAM)

- **RAM:** ist flüchtig und dient der temporären Datenspeicherung während der Programmausführung.
- **SRAM:** speichert Bits in Flipflops, sehr schnell, kein Refresh, aber teuer → Einsatz als Cache oder Registerspeicher.
- **DRAM:** speichert Bits als Ladung im Kondensator, benötigt periodisches Refresh, hohe Dichte → Einsatz als Hauptspeicher.
- **DDR-SDRAM:** überträgt Daten an beiden Taktflanken und erreicht dadurch hohe Bandbreiten.

6.3 NVRAM

Zwischen den klassischen Kategorien flüchtiger und nichtflüchtiger Speicher existiert eine Speicherklasse, die die Vorteile beider Welten vereint: **nichtflüchtige RAMs (NVRAM)**. Diese Speicher behalten ihren Inhalt auch bei fehlender Versorgungsspannung, ermöglichen aber – ähnlich wie herkömmliche RAMs – **schnelle Lese- und Schreibzugriffe mit wahlfreiem Zugriff**. Dadurch eignen sie sich besonders für Anwendungen, bei denen Daten kontinuierlich geändert werden, aber trotzdem erhalten bleiben müssen, etwa in Messsystemen, Datenloggern, Echtzeituhr-Bausteinen oder Steuergeräten mit sicherheitsrelevanten Parametern.

Frühere NVRAM-Konzepte basierten häufig auf der Kombination aus **SRAM und Batterie** (sog. *battery-backed SRAM*). Dabei wird ein statischer RAM mit einer kleinen Lithiumzelle versorgt, um den Speicherinhalt bei ausgeschaltetem System zu erhalten. Diese Lösung ist einfach, aber wartungsintensiv und in modernen Designs aufgrund von Zuverlässigkeits- und Umweltsanforderungen kaum mehr gebräuchlich.

Aktuelle Systeme verwenden **nichtflüchtige Speichertechnologien auf Halbleiterebene**, die ohne Batteriepuffer auskommen. Die wichtigsten Vertreter sind:

- **FRAM (Ferroelectric RAM):**

FRAM verwendet ferroelektrische Materialien (z. B. PZT – Blei-Zirkonat-Titanat), deren Polarisationszustand ein Bit repräsentiert. Das Umklappen der Polarisation erfolgt elektrisch und erfordert keine hohen Spannungen wie bei Flash-Speichern. FRAM kombiniert daher **hohe Schreibgeschwindigkeit, niedrigen Energieverbrauch und hohe Schreibzyklenfestigkeit** (bis zu 10^{14} Zyklen). Nachteilig ist die vergleichsweise geringe Speicherdichte, weshalb FRAM meist für kleinere Datenmengen (z. B. Parameter, Log-Daten) eingesetzt wird.

- **MRAM (Magnetoresistive RAM):**

MRAM speichert Informationen magnetisch, indem es den Widerstand eines magnetischen Tunnelübergangs (MTJ – *Magnetic Tunnel Junction*) verändert. Der Widerstandszustand entspricht einem logischen 0- oder 1-Zustand. MRAM ist **nichtflüchtig**, erlaubt **schnellen, symmetrischen Zugriff** und weist eine sehr hohe Lebensdauer auf. Da der Schreibvorgang jedoch magnetisch erfolgt, erfordert die Technologie komplexere Herstellungsprozesse und spezielle Fertigungsschritte.

- **ReRAM (Resistive RAM) und PCRAM (Phase Change RAM):**

Diese Speicherarten nutzen eine veränderliche Leitfähigkeit im Material (ReRAM) oder einen Phasenwechsel zwischen amorphem und kristallinem Zustand (PCRAM), um logische Zustände darzustellen. Beide Technologien befinden sich teils noch in der Entwicklungs- bzw. Nischenanwendung, versprechen aber hohe Dichte und kurze Zugriffszeiten.

Durch ihre Kombination aus Geschwindigkeit, Energieeffizienz und Nichtflüchtigkeit sind

NVRAM-Technologien ein wesentlicher Bestandteil moderner Embedded- und IoT-Systeme geworden.

So integrieren z. B. Mikrocontroller von Texas Instruments oder Infineon heute **FRAM-basierte Speicherblöcke** für schnelles, energieeffizientes Datenspeichern ohne separate EEPROM-Zugriffe. In sicherheitskritischen Anwendungen (z. B. Automotive, Industrie) ermöglichen NVRAMs das **sichere Speichern von Zustandsdaten** auch bei plötzlichem Spannungsausfall.

Merke: NVRAM

Nichtflüchtige RAMs kombinieren die **Zugriffsgeschwindigkeit von RAM** mit der **Datenerhaltung nichtflüchtiger Speicher**.

- **FRAM:** ferroelektrisch, extrem hohe Schreibzyklenzahl, sehr schnell, geringe Dichte.
- **MRAM:** magnetoresistiv, langlebig, symmetrische Lese-/Schreibzeiten.
- **ReRAM / PCRAM:** nutzen Materialeffekte (Leitfähigkeit bzw. Phasenwechsel), derzeit noch Spezialanwendungen.

NVRAMs eignen sich ideal für **Echtzeit- und Embedded-Systeme**, bei denen Daten häufig geändert, aber dauerhaft erhalten werden müssen.

KI-Aufgabe: FRAM als Ersatz für I²C-EEPROM

Im Zuge eines Re-Designs soll ein herkömmlicher I²C-basierter Speicher, z. B. ein 24C256-EEPROM, durch einen **FRAM-basierten Speicher** ersetzt werden. Bitte eine KI, dir zu erklären, **welche technischen Unterschiede** zwischen beiden Speicherarten bestehen und **was bei einem Austausch zu beachten ist**.

Überlege und recherchiere insbesondere:

- Welche Anpassungen sind hinsichtlich **Schreibzugriff, Zykluszeiten und Adressierung** erforderlich?
- Welche Vorteile entstehen (z. B. unbegrenzte Schreibzyklen, kein Write-Delay)?
- Gibt es Punkte, die softwareseitig berücksichtigt werden müssen (z. B. Wegfall von Wartezeiten, ggf. anderer Speicherumfang)?

Halte die wichtigsten Ergebnisse in **eigenen Worten** fest und diskutiere mit deinem Banknachbarn, **unter welchen Bedingungen** ein 1:1-Austausch möglich wäre und wann **Layout- oder Firmwareanpassungen** erforderlich sind.

6.4 Cache

Der **Cache** ist ein besonders schneller Pufferspeicher zwischen CPU und Hauptspeicher. Seine Aufgabe besteht darin, häufig benötigte Daten und Instruktionen vorzuhalten, um die Zugriffszeit auf den vergleichsweise langsamen Arbeitsspeicher drastisch zu reduzieren. Da moderne Prozessoren Daten mit Gigahertz-Taktraten verarbeiten, während Speicherzugriffe im Bereich von Nanosekunden bis Mikrosekunden liegen, würde ein direkter Zugriff auf den Hauptspeicher die Ausführungsgeschwindigkeit erheblich bremsen. Der Cache kompensiert diese Diskrepanz, indem er auf dem Prinzip der **Lokalität von Speicherzugriffen** basiert.

Man unterscheidet zwei Formen der Lokalität:

- **zeitliche Lokalität (temporal locality):** Wird auf eine Speicheradresse zugegriffen, so ist die Wahrscheinlichkeit hoch, dass dieselbe Adresse bald erneut benötigt wird (z. B. Schleifenvariablen).
- **räumliche Lokalität (spatial locality):** Wenn auf eine bestimmte Adresse zugegriffen wird,

sind benachbarte Adressen mit hoher Wahrscheinlichkeit ebenfalls relevant (z. B. Felder, Arrays, Instruktionsblöcke).

Moderne Prozessoren besitzen oft mehrere Cache-Ebenen:

- Der **L1-Cache** ist direkt in der CPU integriert, extrem schnell, aber sehr klein (typ. 16–128 kB).
- Der **L2-Cache** ist größer (typ. einige 100 kB bis mehrere MB) und etwas langsamer.
- Hochleistungsprozessoren verfügen zusätzlich über einen **L3-Cache**, der meist als gemeinsamer Cache für mehrere CPU-Kerne dient.

Beim Zugriff auf Daten oder Instruktionen prüft der Prozessor, ob sich diese bereits im Cache befinden (**Cache Hit**). Ist dies der Fall, kann der Zugriff mit minimaler Latenz erfolgen. Wenn die benötigten Daten nicht vorhanden sind (**Cache Miss**), werden sie aus dem nächstniedrigeren Speicher (z. B. Hauptspeicher) nachgeladen. Dieser Nachladevorgang kann mehrere hundert Taktzyklen dauern, weshalb eine hohe Trefferquote („Hit Rate“) entscheidend für die Systemleistung ist.

Die Organisation eines Caches wird durch mehrere Parameter bestimmt:

- **Cache-Größe** (Gesamtanzahl der speicherbaren Bytes oder Cache-Lines)
- **Cache-Line-Größe** (z. B. 32, 64 oder 128 Byte – die kleinste übertragene Einheit)
- **Mapping-Verfahren**: bestimmt, wie Hauptspeicheradressen Cache-Blöcken zugeordnet werden
 - *Direct Mapped Cache*: jede Speicheradresse kann nur in genau einer Cache-Line liegen → einfach, aber konfliktanfällig.
 - *Fully Associative Cache*: eine Speicheradresse kann in jeder Cache-Line gespeichert werden → flexibel, aber aufwändig.
 - *Set Associative Cache*: Kompromisslösung – Cache ist in Gruppen (Sets) unterteilt, innerhalb eines Sets freie Zuordnung.
- **Ersetzungsstrategie**: legt fest, welcher Eintrag bei einem Miss überschrieben wird (z. B. LRU – Least Recently Used, FIFO).
- **Schreibstrategie**: definiert, wie Datenänderungen behandelt werden:
 - *Write-Through*: Daten werden gleichzeitig in Cache und Hauptspeicher geschrieben.
 - *Write-Back*: Änderungen erfolgen zunächst nur im Cache und werden später (Lazy Write) in den Hauptspeicher übertragen.

Die Wahl dieser Strategien ist ein Balanceakt zwischen Komplexität, Geschwindigkeit und Datenkonsistenz. Ein Write-Back-Cache ist effizienter, erfordert jedoch Mechanismen zur Gewährleistung der Kohärenz, insbesondere bei Mehrkernsystemen.

Zur quantitativen Bewertung dient häufig die mittlere effektive Zugriffszeit (**Average Memory Access Time, AMAT**):

$$t_{\text{eff}} = t_{\text{Hit}} + P_{\text{Miss}} \cdot t_{\text{MissPenalty}}$$

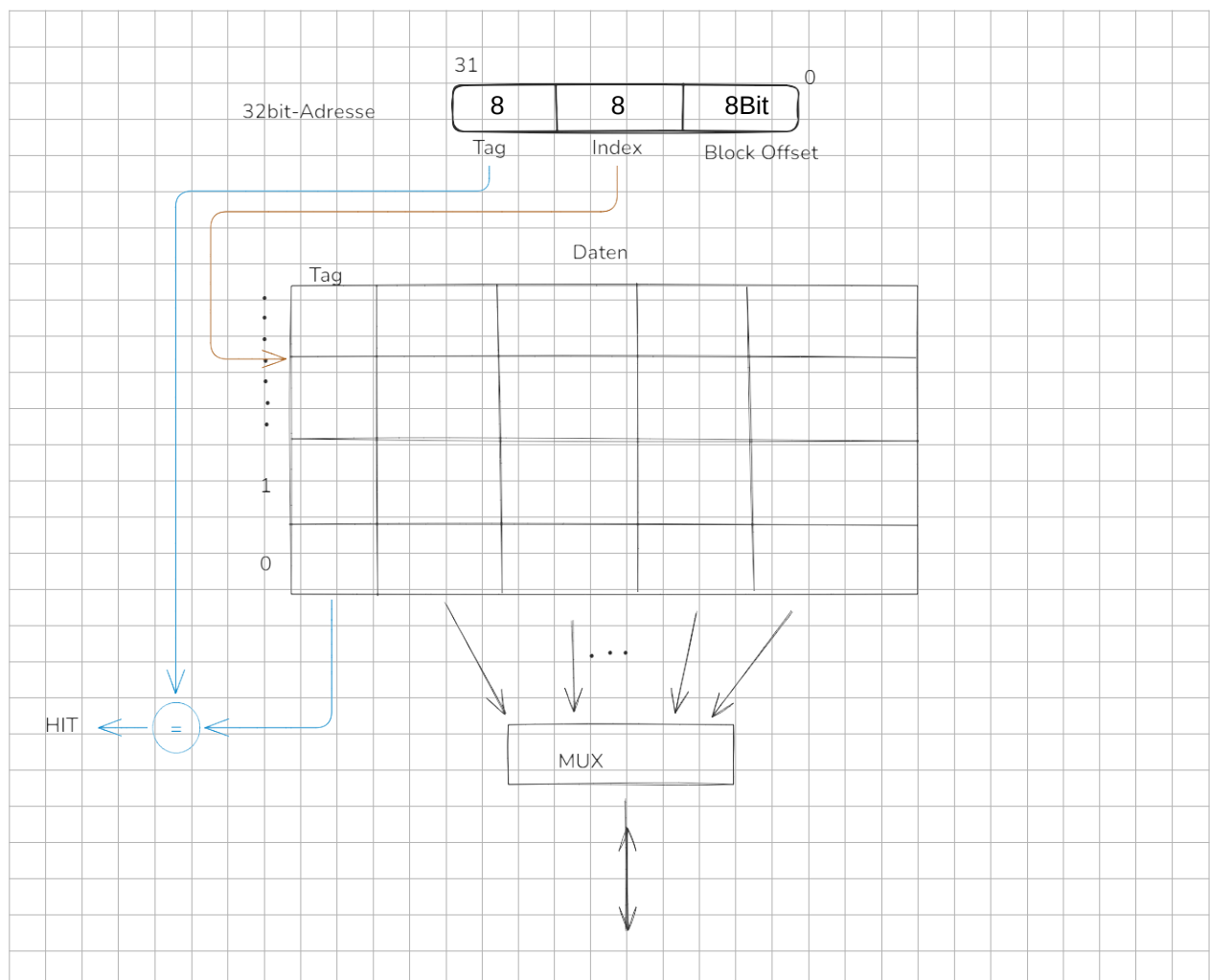
wobei t_{Hit} die Zugriffszeit bei einem Treffer, P_{Miss} die Miss-Rate und $t_{\text{MissPenalty}}$ die Zusatzzeit bei einem Miss beschreibt. Schon kleine Verbesserungen der Hit-Rate führen zu erheblichen Geschwindigkeitsgewinnen.

Aufgabe: Cache – Hit/Miss-Berechnungen

1. Ein Cache hat eine Hit-Rate von 95 %. Die Zugriffszeit im Cache beträgt 2 ns, beim Miss werden 80 ns benötigt. Berechne die mittlere effektive Zugriffszeit t_{eff} .
2. Wie ändert sich t_{eff} , wenn die Hit-Rate auf 98 % steigt? Interpretiere das Ergebnis im Hinblick auf die Systemleistung.
3. Ein zweistufiges Cache-System besitzt folgende Parameter:
 L1: Hit = 1 ns, Miss Rate = 10 %
 L2: Hit = 10 ns, Miss Rate = 20 %, Zugriff auf Hauptspeicher = 100 ns
 Berechne t_{eff} und diskutiere die Wirkung des L2-Caches.

Bei einem **Direct-Mapped Cache** wird jede Speicheradresse genau einer Cache-Line zugeordnet. Die physikalische Speicheradresse wird dazu in drei Felder aufgeteilt:

- **Tag** – enthält die höheren Adressbits zur Identifikation des Speicherblocks,
- **Index** – bestimmt, in welcher Cache-Line der Block abgelegt wird,
- **Block Offset** – wählt das Byte oder Wort innerhalb der Cache-Line aus.



Beim Zugriff liest der Cache die entsprechende Line, vergleicht das gespeicherte Tag mit dem aktuellen Adress-Tag und entscheidet so über **Hit** oder **Miss**. Bei einem Miss wird der

entsprechende Speicherblock aus dem Hauptspeicher nachgeladen, das neue Tag eingetragen und ggf. ein altes überschrieben.

Ein Direct-Mapped Cache ist hardwareseitig einfach aufgebaut, kann aber bei ungünstigen Zugriffsmustern zu sogenannten **Konfliktmisses** führen, da verschiedene Adressen auf denselben Index abgebildet werden können.

Aufgabe: Direct Mapped Cache

Ein 8-Bit-Mikrocontroller besitzt einen **Adressraum von 64 kByte** (16-Bit-Adressierung). Der Cache ist **Direct-Mapped**, besitzt **256 Lines** mit je **1 Byte** Datenkapazität. Damit werden die Adressen folgendermaßen aufgeteilt:

Tag 8 Bit Vergleichsmerkmal zur Identifikation des Speicherblocks
 Index 8 Bit bestimmt die Cache-Line
 Offset 0 Bit entfällt, da jede Line genau 1 Byte enthält

Eine Speicheradresse $A_{15..0}$ wird also als $A_{15..8} = \text{Tag}$, $A_{7..0} = \text{Index}$ interpretiert.

Die folgende Tabelle zeigt die aktuelle Belegung des Arbeitsspeichers:

Adresse	Datum
FFFF	FF
FF03	8C
E000	B1
B127	03 AA
1A2B	0A
83C1	00
55FF	A2
03FF	C2

Die folgende Darstellung zeigt einen Ausschnitt aus der aktuellen Belegung des Cache:

nächster Takt

Index	Tag	Datum
FF	55	A2
C1	83	00
2B	1A	0A
27	B1	03 AA
00	E0	B1

Der Prozessor führt nacheinander folgende **LDA/STA-Befehle** aus:

1. LDA 83C1
2. STA B127 $A = AA$
3. LDA 03FF
4. LDA 03FF
5. STA FFFF $A = 00$
6. LDA 1A2B

- 7. LDA FFFF
- 8. STA FFFF A = 01
- 9. LDA 03FF
- 10. LDA B127
- 11. STA 83C1 A = A0

1. Vervollständige die folgende Tabelle bzgl. hit / miss:

Anweisung	Hit	Miss
LDA 83C1	X	
STA B127 A = AA	X	
LDA 03FF		X
LDA 03FF	X	
STA FFFF A = 00	X	X
LDA 1A2B	X	
LDA FFFF	X	
STA FFFF A = 01	X	
LDA 03FF		X
LDA B127	X	
STA 83C1 A = A0	X	

2. Wie sieht die Belegung des Cache nach folgender Anweisung aus?

1. Anweisung (LDA 83C1)

Index	Tag	Datum
FF	55	A2
C1	83	00
2B	1A	0A
27	B1	03
00	E0	B1

2. Anweisung (STA B127 A = AA)

Indes	Tag	Datum
FF	55	A2
C1	83	00
2B	1A	0A
27	B1	AA
00	E0	B1

3. Anweisung (LDA 03FF)

Index	Tag	Datum
FF	03	A2
C1	83	00
2B	1A	0A

5. Anweisung (STA FFFF A = 00)

Indes	Tag	Datum
FF		
C1		
2B		

27	B1	AA
00	E0	B1

27		
00		

8. Anweisung (STA FFFF A = 01)

Index	Tag	Datum
FF		
C1		
2B		
27		
00		

9. Anweisung (LDA 03FF)

Indes	Tag	Datum
FF		
C1		
2B		
27		
00		

10. Anweisung (LDA B127)

Index	Tag	Datum
FF		
C1		
2B		
27		
00		

11. Anweisung (STA 83C1 A = A0)

Indes	Tag	Datum
FF		
C1		
2B		
27		
00		

4. Wie oft wird bei der Ausführung der Eintrag mit dem Index FF modifiziert?

5. Wie oft wird dabei das Tag verändert?

6. Wie oft wird insgesamt ein Datum im Cache verändert?

 Merke: Cache

Der Cache überbrückt die Geschwindigkeitslücke zwischen CPU und Hauptspeicher, indem häufig benötigte Daten lokal gespeichert werden.

- Prinzip: **Lokalität der Speicherzugriffe** (zeitlich & räumlich).
- Mehrstufiger Aufbau (L1 → L2 → L3).
- Effizienz bestimmt durch Hit/Miss-Rate, Mapping-Verfahren und Schreibstrategie.
- Ziel: Minimierung der **mittleren Zugriffszeit** t_{eff} .